UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería

Departamento de Ciencias de la Computación

CC3092 - Deep Learning y Sistemas Inteligentes

Luis Suriano



Laboratorio 10

Explicabilidad de modelos con Grad-CAM, SHAP, LIME y contrafactuales

Ricardo Godínez - 23247

Vianka Castro - 23201

GUATEMALA, 17 de octubre de 2026

Índice

1. Introducción 2
2. Objetivos 2
Objetivo general 2
Objetivos específicos 2
3. Desarrollo 3
Grad-CAM y CAM sobre ResNet-18 3
SHAP sobre German Credit 4
LIME desde cero 4
Contrafactuales 5
Validación de Grad-CAM 5
4. Resultados 6
Predicciones y mapas de Grad-CAM 6
SHAP sobre el modelo de crédito 9
LIME en los tres clientes 12
Contrafactuales del cliente rechazado 14
Deletion, insertion y cascada 15
5. Discusión 17
Lo que confirmaron los mapas de Grad-CAM 17
Lo que aprendimos de SHAP 19
Lo que aprendimos de LIME y de los contrafactuales 20
Qué entregaríamos a cada audiencia 21
6. Conclusiones 22
7. Referencias 23

1. Introducción

En este laboratorio nos tocó explicar las decisiones de dos modelos muy distintos y, sobre todo, preguntarnos si esas explicaciones merecen confianza. El primero es un ResNet-18 preentrenado en ImageNet, al que le aplicamos Grad-CAM (un método que pinta sobre la imagen las zonas que más empujaron al modelo hacia una clase, como un resaltador sobre un texto). El segundo es un clasificador de gradient boosting entrenado con el conjunto German Credit, que decide si una solicitud de crédito es buena o mala; ahí trabajamos como si fuéramos el equipo de riesgo de modelo de una entidad financiera que tiene que justificar cada decisión.

Para el modelo de crédito usamos tres familias de explicaciones. SHAP reparte la predicción de un cliente entre sus variables usando valores de Shapley de teoría de juegos. LIME, que implementamos desde cero, imita el comportamiento del modelo alrededor de un cliente con una regresión lineal sencilla. Los contrafactuales responden otra pregunta: qué tendría que cambiar el cliente para que lo aprobaran.

La parte que más nos enseñó fue la última: poner a prueba las explicaciones. Comprobamos numéricamente las propiedades que la teoría promete, medimos la estabilidad y la fidelidad de LIME, comparamos métodos entre sí y sometimos los mapas de Grad-CAM a pruebas de borrado de píxeles y de aleatorización de pesos. En el camino encontramos un artefacto del modelo de crédito y una trampa en la forma de medir los mapas, y ambos cambiaron lo que diríamos al entregar estas explicaciones a personas reales. Todo el código lo escribimos con PyTorch, NumPy y scikit-learn, con semilla aleatoria 42.

2. Objetivos

Objetivo general

Explicar las decisiones de un modelo de visión y de un modelo de crédito con las técnicas vistas en clase y evaluar con evidencia numérica si esas explicaciones son confiables.

Objetivos específicos

- Implementar Grad-CAM desde cero sobre la última capa convolucional de un ResNet-18 y verificar su equivalencia numérica con CAM.

- Calcular valores SHAP de un gradient boosting, verificar sus propiedades y programar la fórmula exacta de Shapley para compararla con TreeSHAP.
- Implementar LIME para datos tabulares sin usar la librería lime y medir su estabilidad, su fidelidad y su acuerdo con SHAP.
- Construir explicaciones contrafactuales para un cliente rechazado respetando variables inmutables y accionables.
- Validar los mapas de Grad-CAM con las métricas de deletion e insertion y con la prueba de aleatorización de parámetros en cascada.
- Decidir qué explicación entregaría el equipo a una persona solicitante, a un oficial de cumplimiento y a un regulador.

3. Desarrollo

Grad-CAM y CAM sobre ResNet-18

Cargamos ResNet-18 con los pesos `IMAGENET1K_V1` en modo de evaluación y replicamos el preprocesamiento pedido: lado corto a 256 píxeles, recorte central de 224 × 224 y normalización con la media y la desviación estándar de ImageNet. Elegimos tres imágenes: (a) un pastor alemán centrado, (b) un pastor alemán junto a un gato atigrado y (c) una banana sobre una laptop, un contexto poco habitual para la fruta. Para (c) definimos el recuadro del objeto en `(0, 80, 223, 208)` sobre la imagen de 224 × 224.

Para Grad-CAM registramos dos hooks sobre `layer4` (funciones que PyTorch llama automáticamente al pasar por una capa, como un peaje que anota lo que entra y lo que sale): uno de avance que guarda las activaciones A, de 512 canales de 7 × 7, y uno de retroceso que guarda $\partial y^c / \partial A$. Retropropagamos desde el logit de la clase, antes del softmax, tomamos como peso α_k^c el promedio espacial del gradiente de cada canal y formamos $L^c = \mathrm{ReLU}(\Sigma_k \alpha_k^c A^k)$, que ampliamos a 224 × 224 con interpolación bilineal y normalizamos con su mínimo y su máximo.

```
logits[0, class_index].backward()
alpha = self.gradients.mean(dim=(2, 3), keepdim=True)
raw_7 = (alpha * self.activations).sum(dim=1)[0]
normalized_7 = minmax(F.relu(raw_7))
```

Como ResNet-18 termina en pooling global y una capa lineal, también calculamos CAM directamente con los pesos de `fc`: $M^c_{ij} = \Sigma_k w_k^c A^k_{ij}$, sin gradientes. Con Z = 49 posiciones, el logit

debe poder escribirse como $(1/Z)\ \Sigma_{ij}\ M^c_{ij} + b^c$, y antes de la ReLU y la normalización $Z \cdot L^c$ debe coincidir con M^c. Verificamos las dos cosas para cada imagen.

SHAP sobre German Credit

Cargamos German Credit (1 000 solicitudes y 20 variables, 7 numéricas y 13 categóricas), codificamos `good` como 1 y `bad` como 0, y pasamos las categóricas por un `OrdinalEncoder` ajustado solo con entrenamiento. Dejamos constancia de una limitación: esos códigos son arbitrarios, y al tratarlos como números los árboles pueden hacer cortes sobre un orden que no tiene significado. Dividimos los datos 80/20 de forma estratificada y entrenamos un `GradientBoostingClassifier` con 200 árboles de profundidad 3 y semilla 42.

Calculamos los valores SHAP del conjunto de prueba con TreeSHAP intervencional sobre la salida en log-odds, con un fondo de 100 instancias de entrenamiento elegidas al azar. Verificamos la eficiencia (que el valor base más la suma de los φ iguale la salida del modelo), comparamos la importancia global media $|\varphi|$ con la importancia por permutación, estudiamos qué pasa con una copia ruidosa de la variable numérica más importante, comparamos KernelSHAP con TreeSHAP y, para un modelo de seis variables numéricas, programamos la fórmula exacta de Shapley enumerando las $2^6 = 64$ coaliciones con sus pesos factoriales.

LIME desde cero

Recreamos el mismo modelo del Task 2 y comprobamos con la exactitud y el AUC guardados que era idéntico. La idea de LIME es inventar clientes parecidos al real y ver cómo reacciona el modelo. Generamos $N = 5\ 000$ vectores binarios z' con cada bit igual a 1 con probabilidad 0.5: si el bit vale 1, la variable conserva el valor del cliente; si vale 0, toma un valor al azar de esa columna de entrenamiento. El gradient boosting predice la probabilidad de good de cada cliente inventado, y cada uno recibe un peso $\pi = \exp(-D^2/\sigma^2)$, donde D^2 es el número de variables ausentes y $\sigma = \sigma_rel \cdot \sqrt{20}$ con $\sigma_rel = 0.75$.

Con esos pesos ajustamos una regresión ponderada con regularización usando la solución cerrada $w = (Z'^{T}\Pi Z' + \lambda I)^{-1} Z'^{T}\Pi y$, con $\lambda = 1$ y una columna de unos para el intercepto, que decidimos no penalizar porque representa la probabilidad base y no hay motivo para empujarla a cero. Después conservamos las $K = 5$ variables de mayor $|w|$, volvimos a ajustar solo con ellas y calculamos la fidelidad ponderada R^2_w (qué tan bien la recta imita al modelo en el vecindario). Antes de

interpretar nada comprobamos la implementación contra `sklearn.linear_model.Ridge`, que minimiza la misma función: los coeficientes coincidieron con un error máximo de 2.8×10^{-15} y el intercepto con 9.0×10^{-15}.

```
design = np.column_stack([np.ones(len(Z_binary)), Z_binary])
penalty = lam * np.eye(design.shape[1])
penalty[0, 0] = 0.0  # el intercepto no se penaliza
weighted_design = design.T * weights
w = np.linalg.solve(weighted_design @ design + penalty, weighted_design @ target)
```

Para medir la estabilidad repetimos LIME 20 veces sobre el cliente rechazado en cada combinación de N ∈ {500, 2000, 5000} y σ_rel ∈ {0.5, 0.75, 1.5}, y calculamos el índice de Jaccard promedio entre los conjuntos de las 5 variables principales (la fracción de variables compartidas entre dos listas) y el tamaño efectivo de muestra $N_ef = (\Sigma\pi)^2/\Sigma\pi^2$ (a cuántos clientes de peso completo equivalen los clientes inventados una vez repartidos los pesos, como una votación donde algunos votos valen más que otros).

Contrafactuales

Para el cliente rechazado generamos 20 000 candidatos. Cada uno cambia entre 1 y 3 variables accionables (`duration`, `credit_amount`, `installment_commitment`, `checking_status` y `savings_status`) con valores observados en entrenamiento distintos al actual del cliente, para que toda variable marcada como cambiada cambie de verdad. Las variables `age`, `personal_status` y `foreign_worker` nunca se modifican. Un candidato es válido si su probabilidad de good es al menos 0.5. La distancia suma, para las numéricas, el cambio dividido entre el MAD (la desviación absoluta mediana, que mide qué tan lejos de lo normal está típicamente un cliente y permite sumar meses con dinero), y para las categóricas un 1 por cada categoría cambiada.

Validación de Grad-CAM

Para validar los mapas usamos dos pruebas. En deletion ordenamos los 50 176 píxeles por el valor del mapa y, en 20 pasos, reemplazamos por gris medio la fracción k/20 más importante; si el mapa acierta, la probabilidad de la clase debería caer rápido, así que un AUC_del bajo es mejor. En insertion partimos de la imagen desenfocada con σ = 10 píxeles y devolvemos primero los píxeles más importantes; ahí un AUC_ins alto es mejor. Repetimos ambas con 10 órdenes aleatorios para

saber cuánto de la caída o la subida es mérito del mapa, como comparar cavar donde dice un mapa del tesoro con cavar al azar.

La segunda prueba fue la aleatorización de parámetros en cascada, una prueba de cordura: si destruimos lo que el modelo aprendió y el mapa sigue igual, el mapa no estaba explicando al modelo. Reinicializamos con `reset_parameters()` de PyTorch, de forma acumulada y desde la salida hacia la entrada, `fc`, `layer4`, `layer3`, `layer2`, `layer1` y `conv1`, y en cada etapa calculamos el mapa de 7 × 7 para la clase original y su Spearman con el mapa original.

4. Resultados

Predicciones y mapas de Grad-CAM

La Tabla 1 resume las cinco clases de mayor probabilidad de cada imagen. El modelo reconoció al perro de (a) casi sin dudas, repartió la imagen (b) entre el perro y dos clases de gato, y clasificó la banana de (c) correctamente aun fuera de su contexto habitual.

Tabla 1. *Cinco clases de mayor probabilidad por imagen.*

Posición	(a)	(b)	(c)
1	German shepherd 0.976751	German shepherd 0.401887	banana 0.755744
2	Leonberg 0.010374	tabby 0.108870	bell pepper 0.038470
3	malinois 0.008180	Egyptian cat 0.071321	butternut squash 0.034780
4	kelpie 0.001212	hamper 0.032350	clog 0.031685
5	Tibetan mastiff 0.000539	Eskimo dog 0.024935	hammer 0.011993

En la Figura 1 se ve el mapa de German shepherd sobre la imagen (a): la activación más alta cae en la cabeza y el pecho del perro y disminuye hacia las patas y el fondo.



Figura 1. *Grad-CAM de la clase German shepherd en la imagen (a).*

La imagen (b) era la prueba de si el mapa depende de la clase. Las Figuras 2 y 3 comparan el mapa de German shepherd con el de tabby sobre la misma imagen: el primero se concentra en el perro de la izquierda y el segundo en la cara del gato. Al tomar el 20% de píxeles de mayor valor en cada mapa, la intersección fue de 0 píxeles y la unión de 20 072, es decir, un IoU de 0.

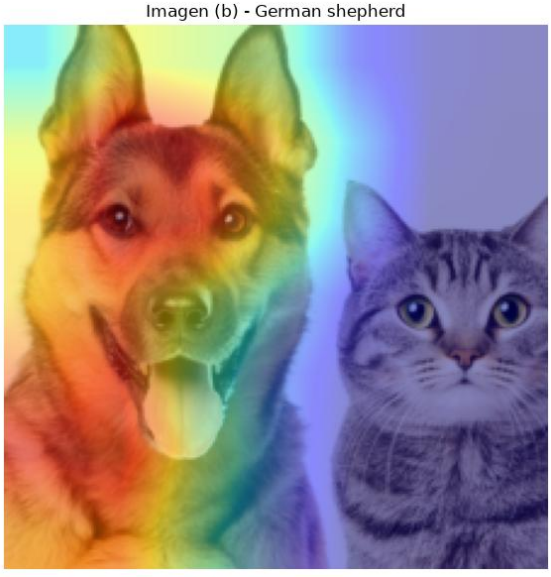


Figura 2. *Grad-CAM de German shepherd en la imagen (b).*

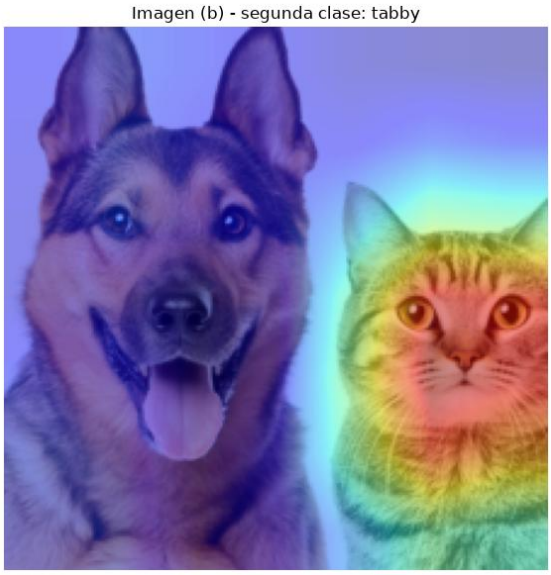


Figura 3. *Grad-CAM de tabby, la segunda clase de la imagen (b).*

La Tabla 2 muestra la verificación numérica de CAM. En las tres imágenes el logit reconstruido con CAM coincide con el directo, $Z \cdot L^c$ coincide con M^c y los mapas normalizados son iguales, con diferencias de 10^{-6} a 10^{-7} que corresponden al redondeo de `float32`. Para los cinco canales de mayor peso de la imagen (a), α_k^c y w_k^c/Z coincidieron con errores relativos de 0, 8.05×10^{-8}, 0, 1.12×10^{-7} y 0.

Tabla 2. *Verificación de la relación entre CAM y Grad-CAM.*

Imagen	Logit	CAM + sesgo	Error del logit	Máx. $\lvert Z \cdot L^c - M^c \rvert$	Máx. error mapas normalizados
(a)	15.6764965057	15.6764974594	9.5367×10^{-7}	7.6294×10^{-6}	1.1921×10^{-7}
(b)	9.4948616028	9.4948616028	0	1.9073×10^{-6}	1.1921×10^{-7}
(c)	11.4739151001	11.4739141464	9.5367×10^{-7}	5.7220×10^{-6}	1.1921×10^{-7}

Para la imagen (c) medimos qué parte del mapa cae dentro del recuadro de la banana y luego hicimos una prueba causal: reemplazamos por gris todo lo que queda fuera del recuadro (Figura 4). El 86.19% de la masa del mapa normalizado cayó dentro del recuadro. Como la región evaluada ocupa el 56.89% de la imagen, un mapa uniforme pondría ahí ese mismo 56.89%, así que la concentración observada es 1.52 veces esa referencia. Al quitar el fondo, la probabilidad de banana pasó de 0.755744 a 0.756964, un cambio de +0.001221.

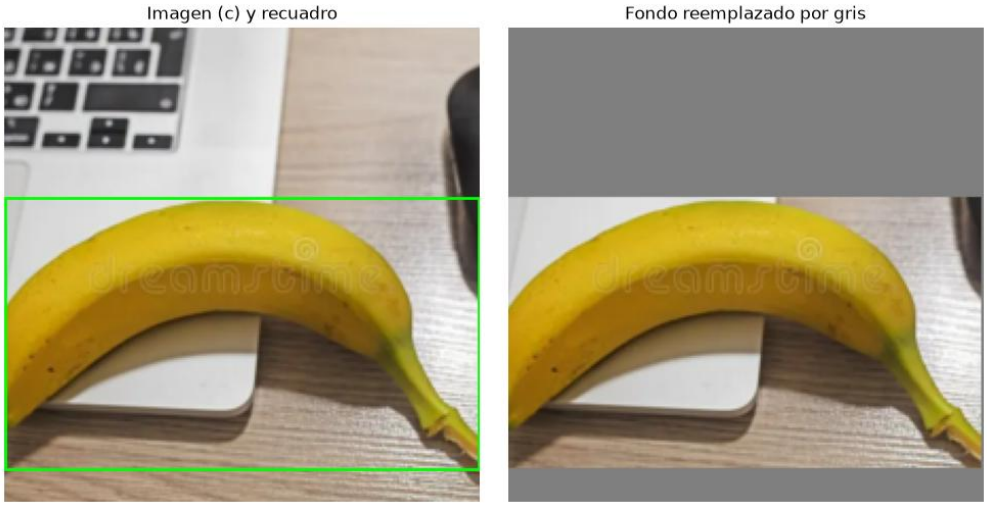


Figura 4. *Imagen (c) con su recuadro y con el fondo reemplazado por gris.*

SHAP sobre el modelo de crédito

El modelo obtuvo una exactitud de 0.7750 y un AUC de 0.7650 en prueba. La eficiencia de SHAP se cumplió con un error máximo de 9.70×10^{-8} en las 200 instancias, y el valor base (1.430742147497 log-odds) coincidió exactamente con el promedio de la salida del modelo sobre el fondo. La Tabla 3 compara las cinco variables principales de cada importancia; la correlación de Spearman entre los dos rankings completos fue de 0.6451.

Tabla 3. *Cinco variables principales por importancia SHAP y por permutación.*

Posición	Media $\|\varphi\|$	I_j	Permutación	Caída media de AUC
1	checking_status	0.760304	checking_status	0.119000
2	purpose	0.511731	credit_amount	0.076405
3	credit_amount	0.490710	duration	0.057905
4	duration	0.391745	credit_history	0.039643
5	savings_status	0.296328	savings_status	0.026155

Elegimos como casos de estudio al cliente de mayor probabilidad de good (índice 175, p = 0.999447) y al de menor probabilidad (índice 721, p = 0.051742), que fue el cliente rechazado de todo el resto del laboratorio. En su cascada (Figura 5) se ve que lo hundieron `credit_history = all paid` (−1.3162 log-odds) y `credit_amount = 433` (−0.9647), mientras que sus ahorros `>=1000` lo empujaron hacia arriba (+0.9721) sin alcanzar a compensar. Para el aprobado, las tres mayores contribuciones fueron `purpose = used car` (+1.4633), `age = 53` (+1.1045) y

`checking_status = no checking` (+1.0069). La Figura 6 muestra la vista global en beeswarm.

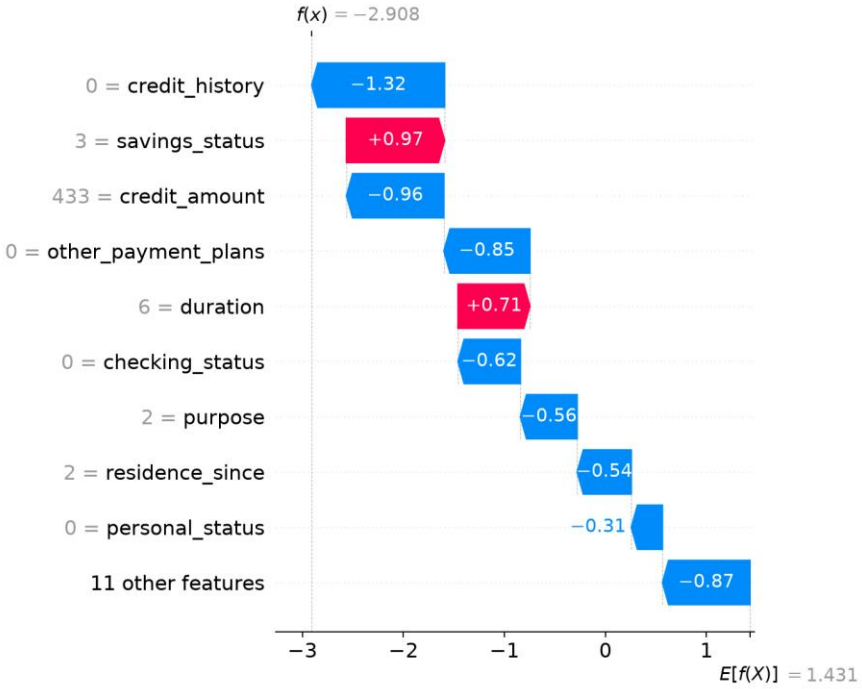


Figura 5. *Cascada SHAP del cliente rechazado (índice 721).*

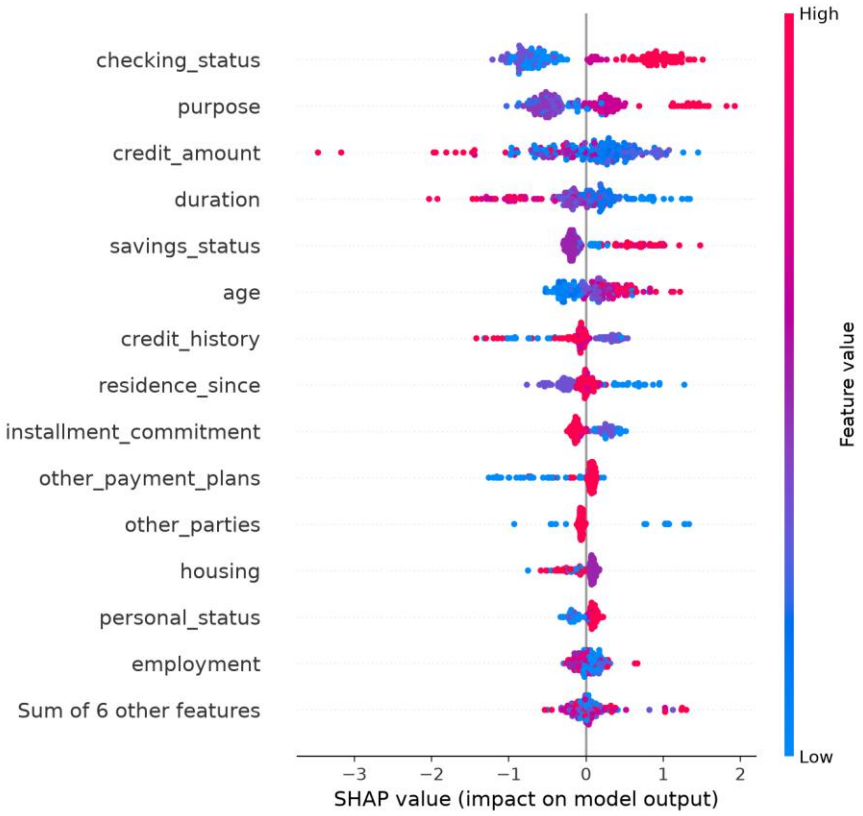


Figura 6. *Gráfico beeswarm de los valores SHAP del conjunto de prueba.*

Para estudiar variables redundantes duplicamos `credit_amount`, la variable numérica de mayor I_j, con un ruido de 0.05σ ($\sigma = 2822.736876$), y reentrenamos con 21 variables. La Tabla 4 muestra cómo se repartió el crédito.

Tabla 4. *Reparto del crédito entre credit_amount y su copia ruidosa.*

Configuración	Media |φ_orig|	Media |φ_copia|	Media |φ_orig + φ_copia|	I_j sin copia
Intervencional	0.518015	0.428365	0.545704	0.490710
Tree path dependent	0.381243	0.362945	0.530267	0.490710

En la comparación de KernelSHAP con TreeSHAP sobre 10 instancias (Tabla 5), el tiempo creció con el fondo, mientras que el error bajó de 10 a 50 instancias y prácticamente no cambió de 50 a 100. Con nuestra propia implementación de la fórmula de Shapley, el error máximo frente a TreeSHAP fue de 2.09×10^{-8} y la eficiencia se cumplió con un error de 2.50×10^{-16}.

Tabla 5. *KernelSHAP con 2 000 muestras por instancia frente a TreeSHAP.*

Fondo	Tiempo total (s)	Tiempo por instancia (s)	MAE frente a TreeSHAP
10	0.8052	0.0805	0.045460
50	2.6448	0.2645	0.040343
100	5.1436	0.5144	0.040586

LIME en los tres clientes

Aplicamos LIME al aprobado, al rechazado y a un cliente de frontera (índice 164, p = 0.503997), el más cercano a 0.5. La Figura 7 muestra los cinco coeficientes de cada uno, que se leen como cuántos puntos de probabilidad sube o baja el cliente por conservar su valor real en lugar de uno tomado al azar. Un detalle que no esperábamos: el cliente aprobado, al que el modelo le da 99.94% de ser good, en realidad es de clase bad.

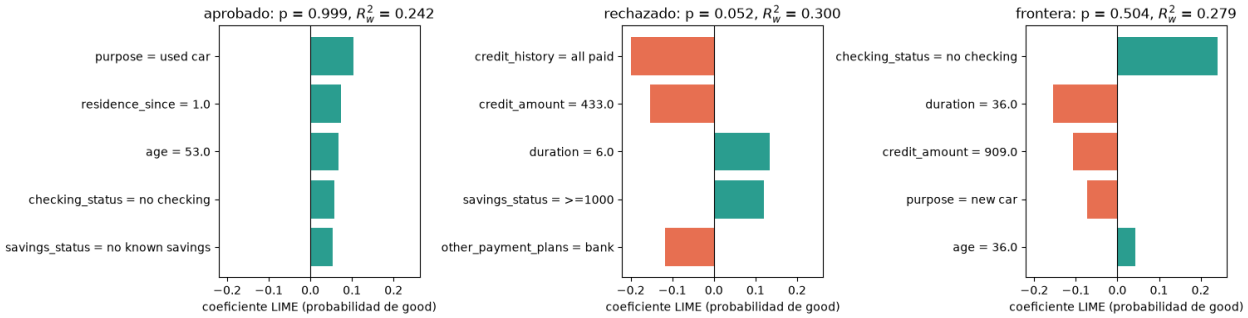


Figura 7. *Coeficientes de LIME (K = 5) para los tres clientes.*

Tabla 6. *Coeficientes y fidelidad de LIME con N = 5 000, K = 5, λ = 1 y σ_rel = 0.75.*

Cliente	p(good)	Variables y coeficientes	R²_w
Aprobado	0.9994	purpose (+0.1033), residence_since (+0.0732), age (+0.0686), checking_status (+0.0567), savings_status (+0.0537)	0.2425
Rechazado	0.0517	credit_history (−0.1986), credit_amount (−0.1544), duration (+0.1335), savings_status (+0.1203), other_payment_plans (−0.1175)	0.2995

Cliente	p(good)	Variables y coeficientes	R^2_w
Frontera	0.5040	checking_status (+0.2408), duration (−0.1538), credit_amount (−0.1064), purpose (−0.0736), age (+0.0438)	0.2790

Las Tablas 7 y 8 resumen la estabilidad sobre el cliente rechazado. Con N = 500 las 20 corridas produjeron 7 listas distintas de variables principales; con N = 5 000 las 20 eligieron exactamente las mismas cinco.

Tabla 7. *Jaccard promedio entre las 5 variables principales (190 pares por celda).*

N	σ_rel = 0.5	σ_rel = 0.75	σ_rel = 1.5
500	0.7927	0.8163	0.7915
2000	0.9368	0.9368	0.9368
5000	1.0000	1.0000	1.0000

Tabla 8. *N_ef promedio en 20 corridas por celda.*

N	σ_rel = 0.5	σ_rel = 0.75	σ_rel = 1.5
500	409.7	480.5	498.8
2000	1639.4	1922.5	1995.1
5000	4099.9	4806.3	4987.6

Al comparar LIME con SHAP (Tabla 9) encontramos bastante acuerdo, sobre todo en el rechazado y en el cliente de frontera, cuyas cinco primeras variables quedaron en el mismo orden con los dos métodos.

Tabla 9. *Acuerdo entre $|w|$ de LIME (20 variables) y $|\varphi|$ de SHAP.*

Cliente	Spearman	Top 3 LIME	Top 3 SHAP	Coinciden
Aprobado	0.7368	purpose, residence_since, age	purpose, age, checking_status	2/3
Rechazado	0.9519	credit_history, credit_amount, duration	credit_history, savings_status, credit_amount	2/3

Cliente	Spearman	Top 3 LIME	Top 3 SHAP	Coinciden
Frontera	0.8692	checking_status, duration, credit_amount	checking_status, duration, credit_amount	3/3

La Figura 8 muestra cómo cambia la fidelidad según el número de variables K que conserva la explicación: 0.2229 con K = 3, 0.2995 con 5, 0.3631 con 10 y 0.3719 con las 20.

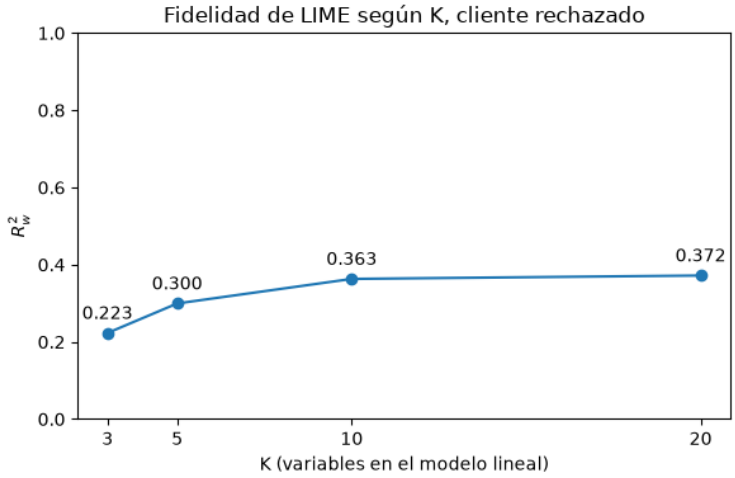


Figura 8. *Fidelidad R²_w de LIME según K en el cliente rechazado.*

Contrafactuales del cliente rechazado

De los 20 000 candidatos, 171 fueron válidos (0.855%). Ninguno de los que cambiaban una sola variable lo fue. La Tabla 10 presenta los tres contrafactuales elegidos; los tres proponen dejar de tener cuenta corriente y, sobre todo, tocar el monto.

Tabla 10. *Contrafactuales del cliente rechazado (p original = 0.0517).*

CF	Cambios	Distancia	p(good)
1	credit_amount 433 → 392; checking_status 0<=X<200 → no checking	1.0388	0.7770
2	duration 6 → 9; credit_amount 433 → 385; checking_status 0<=X<200 → no checking	1.5454	0.6080

CF	Cambios	Distancia	p(good)
3	credit_amount 433 → 426; installment_commitment 4 → 3; checking_status 0<=X<200 → no checking	2.0066	0.7630

La proximidad promedio de los tres fue 1.5303 y la dispersión 2.67 variables cambiadas. Nos llamó la atención que el 100% de los válidos cambiara `credit_amount` y que el primero solo bajara el monto 41 unidades, así que variamos únicamente el monto del cliente (Figura 9). El cliente cae en un valle muy angosto entre 431 y 451, donde la probabilidad es la más baja de todo el rango: con 430 sube a 0.3125 y con 452 a 0.2491. En entrenamiento hay dos clientes bad con montos de 433 y 448 rodeados de clientes good, y los árboles aprendieron cortes en 430.5 y 451 para aislarlos.

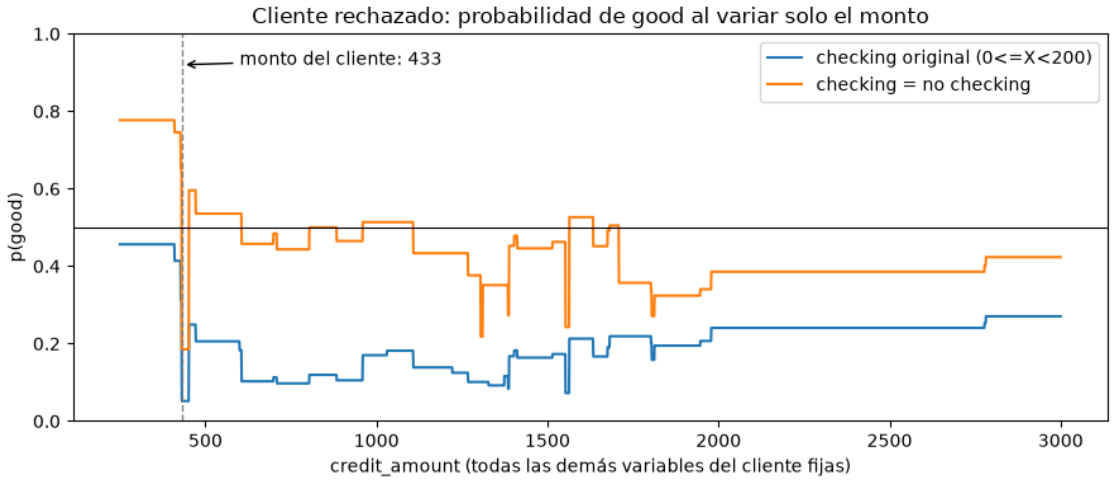


Figura 9. *Probabilidad de good del cliente rechazado al variar solo el monto.*

Deletion, insertion y cascada

La Figura 10 muestra las curvas de deletion e insertion de las tres imágenes frente a la banda del azar, y la Tabla 11 resume sus AUC. La ventaja se expresa en desviaciones estándar de los 10 órdenes aleatorios, en la dirección favorable a Grad-CAM.

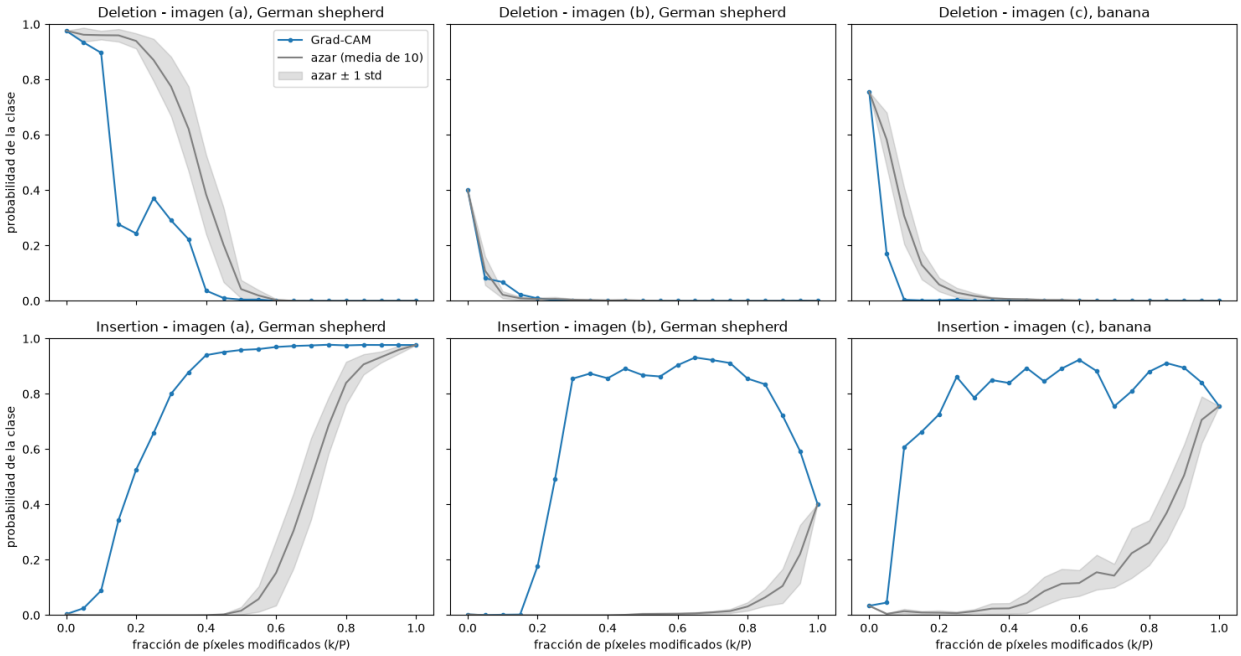


Figura 10. *Curvas de deletion e insertion de Grad-CAM frente al azar.*

Tabla 11. *AUC de deletion e insertion con 20 pasos y 10 órdenes aleatorios.*

Imagen	Métrica	AUC Grad-CAM	Azar (media ± std)	Ventaja (std)
(a)	Deletion	0.1648	0.3372 ± 0.0247	6.98
(a)	Insertion	0.7955	0.3164 ± 0.0291	16.44
(b)	Deletion	0.0093	0.0084 ± 0.0029	−0.32
(b)	Insertion	0.6479	0.0436 ± 0.0081	74.32
(c)	Deletion	0.0092	0.0580 ± 0.0102	4.77
(c)	Insertion	0.7831	0.1794 ± 0.0227	26.57

La Figura 11 muestra los mapas a lo largo de la cascada y la Tabla 12 sus correlaciones con el mapa original. Desde la primera etapa el modelo dejó de reconocer las imágenes: la probabilidad de la clase original cayó a entre 0.0019 y 0.0023, el nivel del azar entre 1 000 clases, y con todo el modelo aleatorizado las tres imágenes se clasificaron como `colobus`.

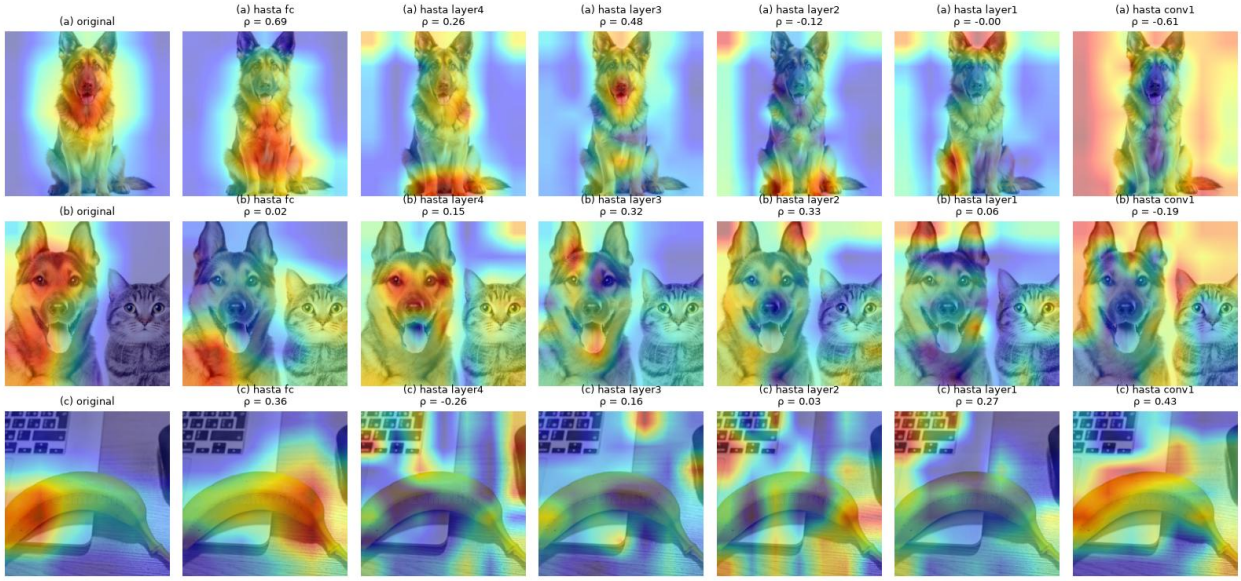


Figura 11. *Mapas de Grad-CAM a lo largo de la aleatorización en cascada.*

Tabla 12. *Spearman entre el mapa de 7 × 7 de cada etapa y el original.*

Aleatorizado hasta	(a)	(b)	(c)
fc	0.6939	0.0173	0.3641
layer4	0.2605	0.1509	−0.2606
layer3	0.4801	0.3177	0.1647
layer2	−0.1220	0.3265	0.0269
layer1	−0.0001	0.0615	0.2690
conv1	−0.6139	−0.1933	0.4253

5. Discusión

Lo que confirmaron los mapas de Grad-CAM

La equivalencia con CAM no fue una casualidad numérica. Como después de `layer4` solo hay un promedio de las 49 posiciones y una capa lineal, el gradiente $\partial y^c/\partial A^k_{ij}$ vale w_k^c/Z en todas las posiciones, y al promediarlo se obtiene ese mismo valor. Los errores que observamos no son cero solo porque `float32` guarda unas siete cifras y el orden de las sumas cambia el redondeo. El IoU de 0 en la imagen (b) mostró que el mapa discrimina entre clases, aunque con un límite claro: cada celda del mapa original cubre 32 × 32 píxeles, así que un objeto más pequeño o dos objetos dentro de la misma celda no se pueden separar. En la imagen (c), el mapa y la prueba causal coincidieron:

la masa se concentró en la banana y quitar el fondo no bajó la probabilidad, de modo que no encontramos evidencia de un atajo por el contexto.

En las pruebas del Task 4, Grad-CAM superó al azar en cinco de las seis comparaciones. Las curvas avanzan en escalones porque el orden de los píxeles hereda los bloques de 32 × 32 del mapa: cada paso del 5% equivale a unas 2.45 celdas. En (a) la probabilidad se mantuvo en 0.897 hasta el 10% y cayó a 0.277 al 15%, cuando desapareció la cabeza. La excepción fue deletion en (b), donde el mapa empató con el azar. Sospechamos del gris y repetimos deletion con la imagen desenfocada como referencia (Figura 12): en (b) el mapa pasó de −0.32 a 3.21 desviaciones a su favor, en (a) de 6.98 a 4.45 y en (c) de 4.77 a 8.94. Con gris, los píxeles dispersos del orden aleatorio funcionan como ruido que el modelo nunca vio en ImageNet y destruyen la predicción casi sin importar dónde caen (0.109 al eliminar el 5% en (b), frente a 0.243 con la referencia desenfocada). La ventaja del mapa se mantiene con las dos referencias, pero su tamaño depende de cuál elijamos.

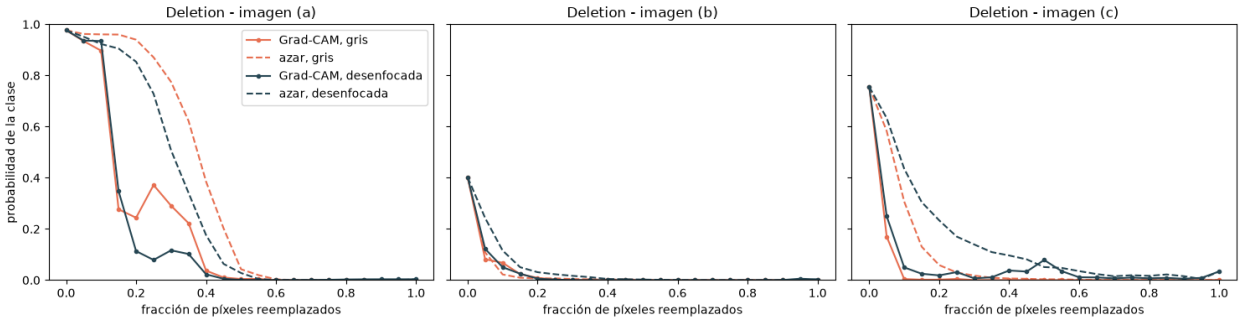


Figura 12. *Deletion con gris y con la imagen desenfocada como referencia.*

En la cascada, el Spearman cayó por debajo de 0.3 en `fc` para (b) y en `layer4` para (a) y (c), pero luego volvió a subir. Para la etapa de `fc` podíamos predecir el resultado: como $\alpha_k^c = w_k^c/Z$ (lo verificamos con un error de 1.75×10^{-10} también con `fc` aleatorio) y las activaciones no cambian, el mapa pasa a ser una combinación con pesos de cualquier signo de los mismos 512 canales, que comparten mucha estructura espacial. Esperábamos un Spearman centrado cerca de cero pero muy disperso. Al repetir esa etapa con 20 semillas (Figura 13) lo confirmamos: (a) dio 0.227 ± 0.470, (b) 0.009 ± 0.427 y (c) 0.117 ± 0.422, y el 0.694 de (a) en la cascada resultó ser una tirada favorable. Grad-CAM depende de los pesos del modelo, pero una cascada con una sola semilla no basta para afirmarlo.

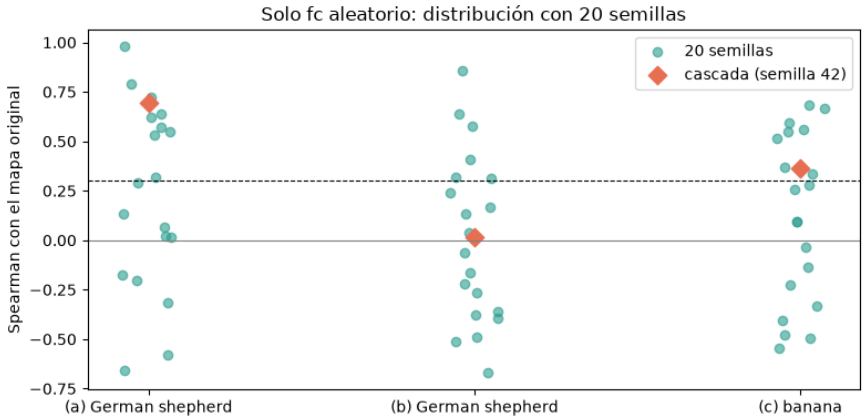


Figura 13. *Spearman con solo fc aleatorio en 20 semillas.*

Lo que aprendimos de SHAP

SHAP suma exactamente en log-odds, que es el espacio que explica, pero no en probabilidad. Con el cliente rechazado, la suma de los efectos individuales llevados a probabilidad fue −0.945719, mientras que el cambio real de p(x) respecto a la base fue −0.755275: cada efecto se calcula partiendo de la probabilidad base de 0.807, donde la sigmoide es empinada, sin tomar en cuenta que al juntarse todos el cliente cae a 0.05, donde la curva ya es plana. La mayor diferencia entre rankings fue `age`, sexta por SHAP (I_j = 0.288582) y vigésima por permutación (−0.005417 ± 0.009607). Sus correlaciones con otras variables fueron moderadas (la más fuerte, −0.3291 con `housing`), así que la hipótesis de redundancia tiene apoyo limitado. Sí queda clara la diferencia de fondo: la permutación mide cuánto empeora el AUC al destruir una variable, mientras que I_j mide el tamaño de su contribución a los log-odds, y una variable puede mover mucho la salida sin mejorar el ordenamiento de los clientes.

Con la variable duplicada, el axioma de simetría (dos jugadores con la misma contribución en toda coalición reciben el mismo valor) no se aplica de forma estricta, porque la copia tiene ruido y los árboles eligen cortes distintos. La versión intervencional le dio más crédito a la original (0.518 frente a 0.428) y la dependiente de la trayectoria lo repartió de forma más pareja (0.381 frente a 0.363), aunque esta última solo aproxima la versión condicional con las frecuencias de las ramas del árbol. La lección es que un valor SHAP describe cómo este modelo, con esta definición de ausencia, reparte su predicción, no la importancia de la variable en el mundo real. En costo, con 0.001161 s por predicción sobre 100 instancias, los 2^{20} valores exactos de un cliente tomarían unos 1 217 s, 2 366 veces más que KernelSHAP; con 60 variables y un microsegundo por evaluación serían unos 36 534 años.

Lo que aprendimos de LIME y de los contrafactuales

La estabilidad de LIME dependió de N y casi nada de σ_rel. Dentro de una misma fila, N_ef cambió de 1639 a 1995 sin que el Jaccard se moviera. Lo que decide la estabilidad es si el ruido de los coeficientes es menor que la separación entre la quinta y la sexta variable (0.0125): con N = 5 000 la desviación de esos coeficientes entre semillas fue de 0.0058 a 0.0076, menor que esa separación; con N = 500 fue de 0.0174 a 0.0206, mayor. Suponiendo que 1 − J decrece como N^(−1/2), estimamos que harían falta unas 1 687 muestras para un Jaccard de 0.9, y 1 101 con una potencia ajustada a nuestros dos puntos (Figura 14). La verificación con 50 semillas dio 0.902 con 1 000, 0.914 con 1 250 y 0.926 con 1 500, aunque 0.845 con 1 100, lo que muestra que el propio Jaccard estimado con pocas corridas tiene ruido. Para tener margen usaríamos N ≥ 1 500.

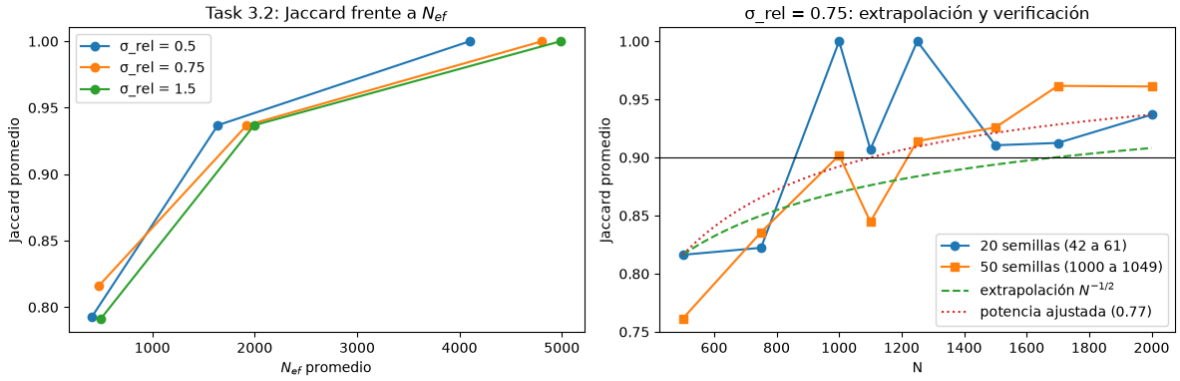


Figura 14. *Jaccard frente a N_ef y verificación de la extrapolación.*

La mayor discrepancia entre LIME y SHAP fue `other_payment_plans = bank` en el cliente aprobado: octava para SHAP (φ = +0.2301) y decimoctava para LIME (w = −0.0006). Nuestra hipótesis fue la saturación de la sigmoide, y la Figura 15 la respalda: pasar de `bank` a `none` cambia los log-odds en 1.36, pero la probabilidad solo en 0.0016, porque el cliente está en p = 0.9994. Reducir σ_rel apenas ayudó (puesto 9 con σ_rel = 0.25, con un coeficiente de solo 0.0066), mientras que correr LIME sobre log-odds la llevó al puesto 8 con w = +0.2362, casi igual al φ de SHAP, y subió el Spearman del cliente de 0.737 a 0.839. La discrepancia se debía a una relación no lineal, no a la localidad ni a una interacción.

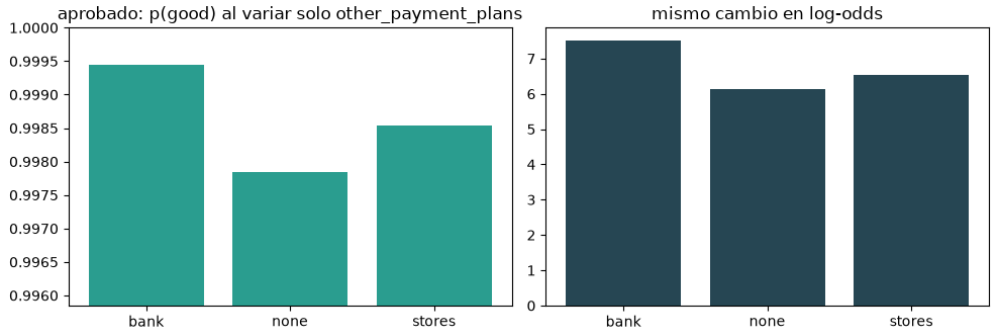


Figura 15. *Cliente aprobado: efecto de other_payment_plans en probabilidad y en log-odds.*

Elegimos K = 5 porque cinco variables se pueden leer de una vez y ya dan el 80.5% del máximo de fidelidad de un modelo lineal. Para una decisión de crédito exigiríamos al menos $R^2_w \geq 0.8$, y no lo alcanzamos con ninguna variante: 0.372 con K = 20, 0.396 explicando log-odds y 0.382 con $\sigma_rel = 0.5$. El techo no viene de que falten variables, sino de que el gradient boosting no es una suma de efectos, y el valle de `credit_amount` es un ejemplo de lo que una recta no puede representar. Por eso no usaríamos LIME como justificación principal.

Los contrafactuales del rechazado solo coinciden con el top 3 de SHAP en `credit_amount`. `credit_history` ($\varphi = -1.3162$) no aparece porque no es accionable, y aun cambiándola sola la probabilidad subiría como máximo a 0.350. `savings_status` (+0.9721) sí es accionable, pero ya ayuda al cliente: su categoría `>=1000` es la mejor posible y cualquier cambio la baja a entre 0.010 y 0.048. Con esto entendimos la diferencia entre las dos preguntas: SHAP responde por qué el modelo dio esta predicción y no la promedio, y el contrafactual responde qué es lo mínimo que tendría que cambiar el cliente para que lo aprueben. El primer contrafactual, sin embargo, es válido para el modelo y absurdo como consejo: aprovecha el valle de 431 a 451, que es un sobreajuste a dos clientes de entrenamiento y no una regla de riesgo.

Qué entregaríamos a cada audiencia

Con todo lo anterior armamos la Tabla 13. La idea que la ordena es que cada explicación se entrega junto con la validación que la respalda, y que lo que no pasó una validación no se entrega como si la hubiera pasado.

Tabla 13. *Explicaciones por audiencia y validaciones que las respaldan.*

Audiencia	Qué entregaríamos	Respaldo numérico	Qué no entregaríamos
Persona solicitante	Las tres razones principales según SHAP en lenguaje claro: historial (−1.32), monto (−0.96) y ahorros a favor (+0.97)	Eficiencia de SHAP con error máximo de 9.7×10^{-8}	Coeficientes de LIME (R^2_w de 0.242 a 0.300), el contrafactual automático de 433 → 392 y consejos sobre variables inmutables
Oficial de cumplimiento	Cascadas SHAP por caso, importancia global y LIME como segunda opinión, más los riesgos del modelo: el valle del monto, el aprobado que es bad, la codificación ordinal y el peso de age	Jaccard de LIME de 1.000 con N = 5 000 y 0.816 con N = 500; Spearman con SHAP de 0.737 a 0.952	LIME sin su R^2_w y resultados de una sola semilla como definitivos
Regulador	Documentación del modelo y de la validación de sus explicaciones, con sus limitaciones	Exactitud 0.775, AUC 0.765, Spearman SHAP y permutación 0.645, Shapley exacto con error de 2.1×10^{-8}, validez de contrafactuales 0.855%	Datos individuales de clientes y explicaciones sin validar

En un sistema de detección de lavado como el del Proyecto 2, que marca secuencias de transacciones sospechosas, un contrafactual sería un riesgo directo: le diría a la persona investigada qué modificar para dejar de ser detectada, por ejemplo partir montos o cambiar el ritmo de sus transferencias. En el modelo de crédito vimos que 41 unidades de diferencia bastaban para cambiar la decisión; en lavado, eso equivale a enseñar a evadir el sistema, y además la regulación prohíbe avisar a la persona de que está siendo reportada. En su lugar entregaríamos explicaciones solo internas para los analistas, como las transacciones que más pesaron en la alerta según la atención o SHAP por transacción, contrastadas con las etiquetas por transacción, y reportes agregados para auditoría.

6. Conclusiones

- Implementamos Grad-CAM desde cero y comprobamos que en ResNet-18 coincide con CAM hasta el redondeo de float32, porque $\alpha_k^c = w_k^c/Z$. El mapa discriminó entre perro y gato (IoU de 0) y no encontramos evidencia de que el modelo usara el fondo para reconocer la banana.

- SHAP cumplió sus propiedades con errores de 10^{-8} o menores, y nuestra fórmula exacta de Shapley coincidió con TreeSHAP. Aun así, aprendimos que sus valores dependen de la definición de ausencia y del fondo, como se vio al repartir el crédito de una variable duplicada.
- LIME resultó estable con 5 000 muestras, pero poco fiel: su recta explica entre el 24% y el 30% del comportamiento del modelo. Coincidió bastante con SHAP en qué variables importan, y su mayor discrepancia se explicó por la saturación de la sigmoide cerca de p = 1.
- Los contrafactuales nos llevaron a descubrir un valle del modelo entre montos de 431 y 451 creado por dos clientes de entrenamiento. Una explicación puede ser válida para el modelo y, al mismo tiempo, inútil o riesgosa para la persona.
- Grad-CAM superó al azar en cinco de seis pruebas y pasó la prueba de aleatorización en promedio, pero los resultados dependieron de la referencia de deletion y variaron mucho con la semilla. Si repitiéramos el laboratorio, haríamos varias corridas desde el inicio en lugar de confiar en una sola.
- La conclusión que atraviesa todo el trabajo es que ninguna explicación se entrega sola: cada una debe ir con la validación que la respalda y adaptada a quién la recibe.

7. Referencias

Adebayo, J., Gilmer, J., Muelly, M., Goodfellow, I., Hardt, M. y Kim, B. (2018). Sanity checks for saliency maps. *Advances in Neural Information Processing Systems 31*.

Adiroot Komsawart. (s. f.). *Happy dog and cat that looking at the camera together isolated on white background* [Fotografía, ID 47006768]. Vecteezy. https://www.vecteezy.com/photo/47006768-happy-dog-and-cat-that-looking-at-the-camera-together-isolated-on-white-background

Dreamstime. (s. f.). *Banana sobre la mesa cerca del ordenador con un ratón de refrigerio* [Fotografía, ID 293587325]. https://es.dreamstime.com/banana-sobre-la-mesa-cerca-del-ordenador-con-un-rat%C3%B3n-de-refrigerio-image293587325

He, K., Zhang, X., Ren, S. y Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 770-778.

Hofmann, H. (1994). *Statlog (German Credit Data)* [Conjunto de datos]. UCI Machine Learning Repository. https://doi.org/10.24432/C5NC77

Lundberg, S. M. y Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems 30*.

Lundberg, S. M., Erion, G., Chen, H., DeGrave, A., Prutkin, J. M., Nair, B., Katz, R., Himmelfarb, J., Bansal, N. y Lee, S.-I. (2020). From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence, 2*, 56-67.

Magnific. (s. f.). *Dog is sitting, white background, with black brown face* [Fotografía premium, ID 369849084]. https://www.magnific.com/premium-photo/dog-is-sitting-white-background-with-black-brown-face_369849084.htm

Paszke, A. et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems 32*.

Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825-2830.

Petsiuk, V., Das, A. y Saenko, K. (2018). RISE: Randomized input sampling for explanation of black-box models. *Proceedings of the British Machine Vision Conference*.

Ribeiro, M. T., Singh, S. y Guestrin, C. (2016). "Why should I trust you?": Explaining the predictions of any classifier. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1135-1144.

Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D. y Batra, D. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. *Proceedings of the IEEE International Conference on Computer Vision*, 618-626.

Wachter, S., Mittelstadt, B. y Russell, C. (2018). Counterfactual explanations without opening the black box: Automated decisions and the GDPR. *Harvard Journal of Law & Technology, 31*(2), 841-887.

Zhou, B., Khosla, A., Lapedriza, A., Oliva, A. y Torralba, A. (2016). Learning deep features for discriminative localization. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2921-2929.